



Formalizing Keyed-Verification Anonymous Credentials in Lean

The μ CMZ credential system

July 30th, 2026

The μ CMZ credential system

Keyed-verification credential μ CMZ

Base MAC

Procedure $\mu\text{CMZ.S}(1^\lambda, n)$	Procedure $\mu\text{CMZ.K}(\text{crs})$	Procedure $\mu\text{CMZ.M}((x_0, x_r, \vec{x}), \vec{m})$
$(\mathbb{G}, p, G) := \Gamma \leftarrow \text{GrGen}(1^\lambda)$	$sk := (x_0, x_r, x_1, \dots, x_n) \leftarrow \mathbb{Z}_q^{n+2}$	$U \leftarrow \mathbb{G}$
$H \leftarrow \mathbb{G}$	$X_0 := x_0 H; \quad X_r := x_r G$	$V := (x_0 + x_r + \sum_i m_i x_i) U$
return $\text{crs} := (\Gamma, H)$	$X_i := x_i G \quad \text{for } i \in [n]$	return $\sigma := (U, V)$
	return $sk, pp := (X_0, X_r, \vec{X})$	
		Procedure $\mu\text{CMZ.V}((x_0, x_r, \vec{x}), \vec{m}, (U, V))$
		return $U \neq 0G \wedge V = (x_0 + x_r + \sum_i m_i x_i) U$

Credential Issuance

$\mu\text{CMZ.I.Usr}((X_0, X_r, \vec{X}), \vec{m}, \phi)$	$\mu\text{CMZ.I.Srv}((x_0, x_r, \vec{x}), \phi)$
$C' := \sum_i m_i X_i + sG$ where $s \leftarrow \mathbb{Z}_p$	
$\pi_{iu} \leftarrow \text{ZKP}_{\text{cmz.iu}}.P((C', \vec{X}, \phi), (\vec{m}, s))$	
	$C'' := C' + X_r$
	$U' := uG$ where $u \leftarrow \mathbb{Z}_p$
	$V' := x_0 U' + u C''$
	$\pi_{is} \leftarrow \text{ZKP}_{\text{cmz.is}}.P((X_0, C'', U', V'), (x_0, u))$
check $U' \neq 0G$	
check $\text{ZKP}_{\text{cmz.is}}.V((X_0, C' + X_r, U', V'), \pi_{is})$	
$U := rU'$ where $r \leftarrow \mathbb{Z}_p$	
$V := r(V' - sU')$	
return $\sigma := (U, V)$	

Credential Presentation

$\mu\text{CMZ.P.Usr}((X_0, X_r, \vec{X}), \vec{m}, (U, V), \phi)$	$\mu\text{CMZ.P.Srv}((x_0, x_r, \vec{x}), \phi)$
$(U', V') := (rU, rV)$ where $r \leftarrow \mathbb{Z}_p$	
$C_V := V' + r'H$ where $r' \leftarrow \mathbb{Z}_p$	
for $i \in [n]$	
$C_i := m_i U' + r_i G$ where $r_i \leftarrow \mathbb{Z}_p$	
$Z := \sum_{i=1}^n r_i X_i - r'H$	
$\pi_p \leftarrow \text{ZKP}_{\text{cmz.p}}.P((U', \vec{X}, \vec{C}, Z, \phi), (r', \vec{r}, \vec{m}))$	
	$Z := (x_0 + x_r)U' + \sum_{i=1}^n x_i C_i - C_V$
	return $(U' \neq 0G \wedge$
	$\text{ZKP}_{\text{cmz.p}}.V((U', \vec{X}, \vec{C}, Z, \phi), \pi_p))$

- **Base MAC** — the algebraic MAC underneath. Construction and perfect correctness machine-checked; Theorem 5.1's AGM unforgeability stated, reduction in progress.
- **Credential Issuance** — blind issuance of a tag on committed attributes. The box's proof relations and Σ -protocols machine-checked; the protocol flow in progress.
- **Credential Presentation** — anonymous showing under a policy ϕ . The box's proof relation R_p (Eq. 11) and its Σ -protocol machine-checked; the flow in progress.

Paper references (equations, figures, pages, theorem numbers) follow the ePrint 2024/1552 revision of 2026-03-27.

Base MAC · the structure

Base MAC		
Procedure $\mu\text{CMZ}.S(1^\lambda, n)$ $(\mathbb{G}, p, G) := \Gamma \leftarrow \text{GrGen}(1^\lambda)$ $H \leftarrow \mathbb{G}$ return $\text{crs} := (\Gamma, H)$	Procedure $\mu\text{CMZ}.K(\text{crs})$ $sk := (x_0, x_r, x_1, \dots, x_n) \leftarrow \mathbb{Z}_q^{n+2}$ $X_0 := x_0 H; \quad X_r := x_r G$ $X_i := x_i G \quad \text{for } i \in [n]$ return $sk, pp := (X_0, X_r, \vec{X})$	Procedure $\mu\text{CMZ}.M((x_0, x_r, \vec{x}), \vec{m})$ $U \leftarrow \mathbb{G}$ $V := (x_0 + x_r + \sum_i m_i x_i) U$ return $\sigma := (U, V)$ Procedure $\mu\text{CMZ}.V((x_0, x_r, \vec{x}), \vec{m}, (U, V))$ return $U \neq 0G \wedge V = (x_0 + x_r + \sum_i m_i x_i) U$

Ambient context: the fixed group

```
variable {F : Type} [Field F] [Fintype F] [DecidableEq F] [SampleableType F]
variable {G : Type} [DecidableEq G] [SampleableGroup F G]
```

Abbreviations used in the carrier types

```
abbrev Key      (F : Type) (n : ℕ) : Type := F × F × (Fin n → F)
abbrev Params  (G : Type) (n : ℕ) : Type := G × G × (Fin n → G)
abbrev Code     (G : Type)      : Type := G × G
```

The carrier types (Crs, Msg, Sk, Pp, Tag) and the four procedures

```
noncomputable def μCMZBaseMACSyntax (gen : G) :
  AlgebraicMACSyntax ProbComp where
  Crs := fun _ => G
  Msg := fun _ => F
  Sk  := fun {_secParam n} _ => Key F n
  Pp  := fun {_secParam n} _ => Params G n
  Tag := fun _ => Code G
  DecidableEqMsg := fun _ => inferInstance
  setup := setup
  keygen := fun {_secParam _} crs => keygen crs gen
  MAC := fun {_secParam _} _ sk m => mac sk m
  verify := fun {_secParam _} _ sk m t => verify sk m t
```

The μCMZ MAC: an instance of the AlgebraicMAC structure (O24 Def 3.1)

```
noncomputable def μCMZBaseMAC (gen : G) : AlgebraicMAC :=
  (μCMZBaseMACSyntax F gen, μCMZBaseMAC_correct F gen)
```

Base MAC · μ CMZ.S (setup)

Procedure μ CMZ.S($1^\lambda, n$)

 $(\mathbb{G}, p, G) := \Gamma \leftarrow \text{GrGen}(1^\lambda)$ $H \leftarrow_{\$} \mathbb{G}$ **return** $\text{crs} := (\Gamma, H)$

μ CMZ.S · setup

```
noncomputable def setup {G : Type}
  [SampleableType G] (_secParam _n : ℕ) :
  ProbComp G :=
  $t G
```

- The formalization is per-group, not asymptotic: each security theorem is an advantage inequality at one arbitrary fixed group, e.g. Theorem 5.1's $\text{Adv}^{\text{ufcmva}}(A) \leq \text{Adv}^{3\text{-dl}}(B_1) + \text{Adv}^{\text{dl}}(B_2) + 3/p$. No λ , no GrGen sampling, no negligibility; at ristretto255 the statistical term is $3/p \approx 2^{-250}$.
- Why: O24 states Theorem 5.1 (with Lemmas 5.4, 5.5 behind it) over GrGen, yet every reduction in the proofs takes Γ as input and argues at that fixed Γ (§5.3). Averaging over Γ recovers the GrGen statements; a deployment instantiates its one group ([full analysis](#)).
- The technical cost of λ : a λ -parameterized protocol samples $\Gamma \leftarrow \text{GrGen}(1^\lambda)$, and a type cannot be sampled inside `ProbComp`, so groups would stop being types and become parameters drawn from a λ -indexed universe of descriptions with an interpretation function. Every carrier and statement would depend on the sampled description, and the reuse the ambient typeclass gives (one `SampleableGroup F G` context shared by all definitions and proofs) would be lost.

Base MAC · μ CMZ.K (keygen)

Procedure μ CMZ.K(crs)

$sk := (x_0, x_r, x_1, \dots, x_n) \leftarrow \$ \mathbb{Z}_q^{n+2}$

$X_0 := x_0 H; \quad X_r := x_r G$

$X_i := x_i G \quad \text{for } i \in [n]$

return $sk, pp := (X_0, X_r, \vec{X})$

μ CMZ.K · keygen

```
noncomputable def keygen {n : ℕ}
  (H : G) (gen : G) :
  ProbComp (Key F n × Params G n) := do
  let x0 ← $t F
  let xr ← $t F
  let x  ← $t (Fin n → F)
  let X0 := x0 • H
  let Xr := xr • gen
  let X  := fun i => x i • gen
  pure ((x0, xr, x), (X0, Xr, X))
```

- The paper's single draw $sk \leftarrow \$ \mathbb{Z}_q^{n+2}$ (Figure 9's notation; the surrounding text writes $\mathbb{Z}_p$) is three draws here: x_0 , x_r , and the vector x , jointly uniform on Key F n.
- $X_0 = x_0 \cdot H$ uses the CRS element H , while X_r and X use `gen` (the paper's generator G ; Lean names it `gen` because G is the carrier type), matching the paper's two base roles.

Base MAC · μ CMZ.M (mac)

Procedure μ CMZ.M($(x_0, x_r, \vec{x}), \vec{m}$)

$U \leftarrow \$ \mathbb{G}$

$V := (x_0 + x_r + \sum_i m_i x_i)U$

return $\sigma := (U, V)$

μ CMZ.M · mac

```
noncomputable def mac {n : ℕ} (sk : Key F n)
  (m : Fin n → F) : ProbComp (Code G) := do
  let U ← uniformNonzero G
  pure (U, macScalar sk m • U)
```

The shared scalar

```
def macScalar {n : ℕ} (sk : Key F n)
  (m : Fin n → F) : F :=
  let (x0, xr, x) := sk
  x0 + xr + ∑ i, x i * m i
```

- Base MAC writes $U \leftarrow \$ \mathbb{G}$; μ CMZ's defining equation (Eq. 6, §2.3.1, p. 16) writes $U \leftarrow \$ \mathbb{G}^\times$. The formalization follows Eq. 6, reading $\mathbb{G}^\times$ as the nonzero elements (explicit in CMZ14: $u \in G \setminus \{1\}$, p. 4): `uniformNonzero G`.
 - Nonzero sampling removes the $1/p$ mass where the honest tag $(0, 0)$ would fail verification: every tag in `mac`'s support verifies. The cost: the signing oracle's distribution changes, so Lemma 5.4's counting argument must be redone under it (deferred, groundwork in `SignMask`; see [design decisions](#)). `verify` still checks $U \neq 0$ against adversarial tags (next slide).
- One definition, `macScalar`, is called by both sides: `mac` returns `(U, macScalar sk m • U)`, `verify` tests `V = macScalar sk m • U`; no drift possible.

Base MAC · μCMZ.V (verify)

Procedure $\mu\text{CMZ.V}((x_0, x_r, \vec{x}), \vec{m}, (U, V))$

return $U \neq 0G \wedge V = (x_0 + x_r + \sum_i m_i x_i)U$

μCMZ.V · verify

```
def verify {n : ℕ} (sk : Key F n)
  (m : Fin n → F) (t : Code G) : Bool :=
  let (U, V) := t
  decide (U ≠ 0) && decide (V = macScalar sk m • U)
```

- The mapping is 1-1: `decide` realizes the two checks as a Bool conjunction; deterministic, so no `ProbComp` .

Base MAC · unforgeability (Theorem 5.1)

O24 Theorem 5.1 (p. 35)

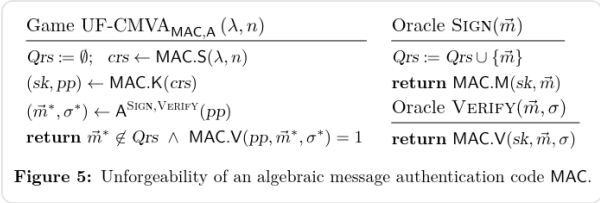
Theorem 5.1. *In the algebraic group model, μCMZ is an algebraic MAC for $n = \text{poly}(\lambda)$ attributes over $\mathbb{Z}_p$ with advantage:*

$$\text{Adv}_{\mu\text{CMZ}}^{\text{ufcmva}}(\lambda, n) \leq \text{Adv}_{\text{GrGen}}^{3\text{-dl}}(\lambda) + \text{Adv}_{\text{GrGen}}^{\text{dl}}(\lambda) + \frac{3}{p},$$

The AGM unforgeability game and advantage (`AlgebraicMAC.lean`)

```
noncomputable def AGM_UF_CMVAGame (secParam : ℕ) (A : AGMUFAdversary F G n) :
  ProbComp Bool := do
  let mac := μCMZBaseMACSyntax F gen
  let H : G ← mac.setup secParam n
  let (sk, pp) ← (mac.keygen (secParam := secParam) (n := n) H :
    ProbComp (Key F n × Params G n))
  let ((mStar, σStar, pU, pV), log) ←
    (simulateQ (agmOracleImpl (gen := gen) secParam sk H pp) (A.run H pp)).run []
  let tags := log.map Prod.snd
  let consistent :=
    pU.eval (gen) H pp.1 pp.2.1 pp.2.2 tags = σStar.1 ∧
    pV.eval (gen) H pp.1 pp.2.1 pp.2.2 tags = σStar.2
  let fresh := mStar ∉ log.map Prod.fst
  pure (decide consistent && decide fresh &&
    mac.verify (secParam := secParam) H sk mStar σStar)

noncomputable abbrev AGM_UF_CMVAAdv (A : AGMUFAdversary F G n) (secParam : ℕ) : ℝ≥0∞ :=
  Pr[= true | AGM_UF_CMVAGame (gen := gen) secParam A]
```



- The proof is set in the algebraic group model: every group element the adversary outputs carries a representation over the elements it has seen, and the forgery must be transcript-consistent and fresh.
- **The game shown is the stronger one O24 proves:** its oracles include the paper's Help oracle (§5.3), whose strength the credential-level proofs (Theorem 5.2's extractability via 5.11) consume downstream — proving it once spares every consumer a separate argument.
- **Scope, stated up front.** The theorem is about this AGM game; the bridge to the plain UF-CMVA game of `KVAC.Core` is a separate, tracked deliverable (its own blueprint node) — sequenced, not forgotten.

Everything on this slide is merged. main carries zero sorry's and zero axiom declarations — Lean's kernel accepts the library from `propext / Classical.choice / Quot.sound` alone — and CI rebuilds every PR before it lands.

Theorem 5.1 · what the analysis corrected in the paper

The corrected target, from constructing the reduction

$$\begin{aligned} \text{Adv}^{\text{ufcmva}}(A) &\leq \text{Adv}^{3\text{-dl}}(B_1) + \text{Adv}^{\text{gap-dl}}(B_{\text{gap}}) + 5/p \\ 5/p &= 3/p \text{ (Schwartz–Zippel, degree 3)} + 1/p \text{ (keygen shear)} + 1/p \text{ (gap-DL denominator)} \end{aligned}$$

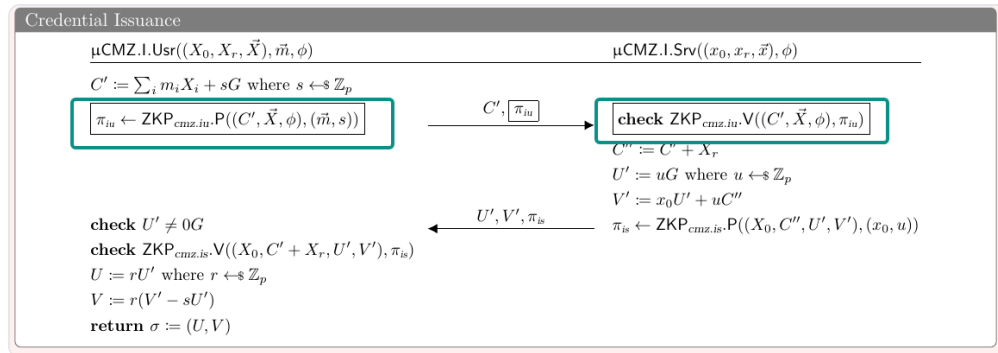
Theorem 5.1. *In the algebraic group model, μCMZ is an algebraic MAC for $n = \text{poly}(\lambda)$ attributes over $\mathbb{Z}_p$ with advantage:*

$$\text{Adv}^{\text{ufcmva}}_{\mu\text{CMZ}}(\lambda, n) \leq \text{Adv}^{3\text{-dl}}_{\text{GrGen}}(\lambda) + \text{Adv}^{\text{dl}}_{\text{GrGen}}(\lambda) + \frac{3}{p} \text{ ,}$$

- **The 1/p non-vanishing bound (p. 38).** The paper asserts the substituted forgery polynomial ψ stays nonzero "except with probability 1/p" — stated bare, no derivation. The bad event is the hidden shift landing on a root of ϕ , and ϕ has total degree 3 (a degree-2 representation term times the degree-1 key polynomial); Schwartz–Zippel gives 3/p. A 1/p bound is what degree 1 would give.
- **Eq. 13 coefficient.** X_0 's X-coefficient is printed $a_h b_0 + b_h$; the correct coefficient is $a_h b_0 + a_0 b_h$ (documented at its use site in the open PR #88 diff).
 - Definitions (Eqs. 12, 16): $\eta = \log_G H = a_h + x \cdot b_h$, and $x_0 = \log_H X_0 = a_0 + x \cdot b_0$.
 - Expansion: $\log X_0 = x_0 \cdot \eta = (a_0 + x \cdot b_0)(a_h + x \cdot b_h) = a_0 a_h + x \cdot (a_0 b_h + b_0 a_h) + x^2 \cdot (b_0 b_h)$.
 - The printed constant and x^2 coefficients match this product; only the x coefficient is off.
- **Eq. 14 coefficient.** A second misprint, in V_j 's G-coefficient: printed $a_u(a_h a_0 + a_h + a_1 m_j)$; the correct factor is $A = a_0 + a_r + a_1 m_j$ (also in PR #88).
 - Definitions: $\text{key}_j = x_0 + x_r + m_j x_1 = A + x \cdot B$ with $A = a_0 + a_r + a_1 m_j$, $B = b_0 + b_r + b_1 m_j$; and $u_j = a_u + x \cdot b_u$.
 - Expansion: $\log V_j = \text{key}_j \cdot u_j = (A + x \cdot B)(a_u + x \cdot b_u) = A \cdot a_u + x \cdot (A \cdot b_u + B \cdot a_u) + x^2 \cdot (B \cdot b_u)$.
 - The printed x and x^2 coefficients use A and B correctly; only the G coefficient garbles A .
- **The gap-DL term.** The printed bound elides the gap-DL advantage its own proof (Lemma 5.5 via Claim 5.6) consumes; we keep it explicit.

Each item is checkable by hand against the paper, and none weakens the result: the assumptions are the paper's own and the constant moves from 3/p to 5/p — $\approx 2^{-250}$ at ristretto255 either way. Full write-up: [errata note](#).

Credential Issuance · R_{iu} relation



R_{iu} — the user's proof: opening of C' , policy ϕ

$$R_{cmz.iu} := \left\{ ((C', \vec{X}, \phi), (\vec{m}, s)) : C' = \sum_i m_i X_i + sG \wedge \phi(\vec{m}) = 1 \right\} \quad (9)$$

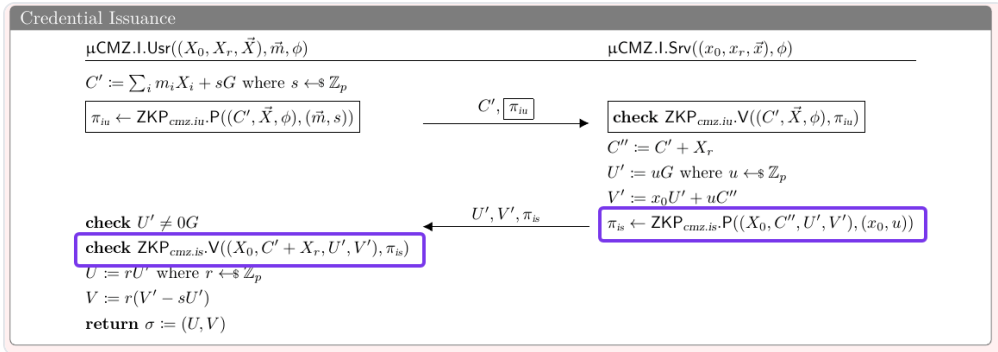
```

def riuRel (gen : G) : RiuStmt G F n → RiuWitness F n → Bool :=
  fun (Cp, X, ϕ) (m, s) => decide (Cp = (∑ i, m i • X i) + s • gen) && ϕ m

```

- **R_{iu} in the box.** The boxed π_{iu} of the panel: the user proves R_{iu} ($ZKP_{cmz.iu}.P$) with witness (m, s) when sending the commitment C' ; the server checks it before issuing.
 - The frame around π_{iu} marks the lines deleted in the anonymous-token variant μCMZ_{AT} ; Theorem 5.3 analyzes that variant's one-more unforgeability (to be presented at a later time).

Credential Issuance · R_{js} relation



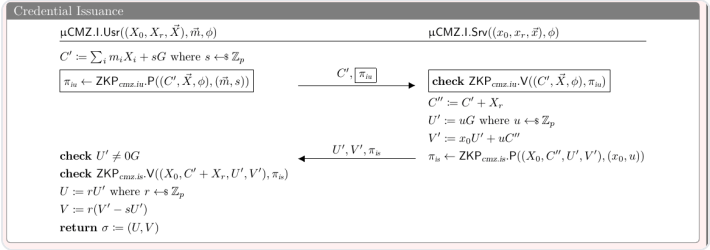
R_{is} — the server's proof: knowledge of (x_0, u) satisfying the displayed equations

$$\mathbf{R}_{cmz.is} := \left\{ ((X_0, C'', U', V'), (x_0, u)) : \begin{array}{l} U' = uG \wedge X_0 = x_0H \wedge \\ V' = x_0U' + uC'' \end{array} \right\} \quad (10)$$

```
def risRel (gen H : G) : RisStmt G → RisWitness F → Bool :=
  fun ⟨X₀, C', U', V'⟩ ⟨x₀, u⟩ => decide
    (U' = u • gen ∧ X₀ = x₀ • H ∧
     V' = x₀ • U' + u • C')
```

- **R_{IS} in the box.** The proof π_{is} on the return leg: the server proves R_{IS} with witness (x_0, u) when sending (U', V') ; the user checks it before unblinding. The relation takes C'' as a statement component and does not prove how it was formed; the user supplies $C'' = C' + X_r$, recomputed locally, which is what binds the proof to this session's commitment.
 - Scope: R_{IS} speaks for x_0 and u only; no issuance proof covers X_r or the X_i .

Credential Issuance · Special Soundness



Our theorem: special soundness with policy enforcement

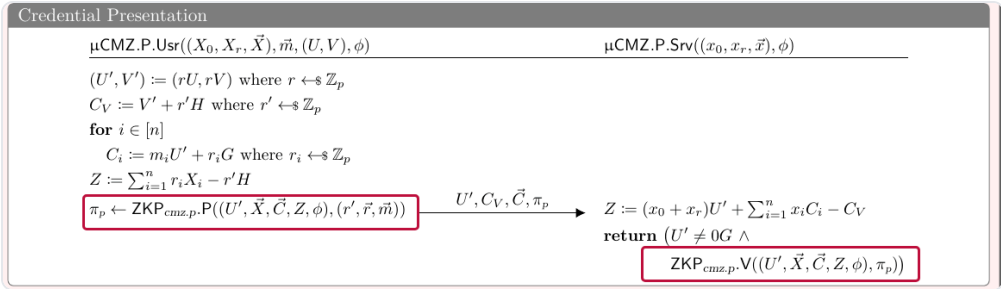
```
theorem riuSigma_speciallySoundAt (gen : G) (Cp : G) (X : PublicBases G n)
  (φ : Policy F n)
  (hφ : Enforces (riuSigma (F := F) (n := n) gen) (Cp, X, φ)
    (fun ⟨m, _s⟩ => φ m)) :
  SpeciallySoundAt (riuSigma (F := F) (n := n) gen) (Cp, X, φ)
```

The dual theorem for R_{IS}: special soundness at every statement

```
theorem risSigma_speciallySound (gen H : G) :
  SpeciallySound (risSigma (F := F) gen H)
```

- Special soundness is proven for `riuSigma` and `risSigma`, the Σ -protocols for R_{IU} and R_{IS}:
 `riuSigma_speciallySoundAt` and `risSigma_speciallySound` (both above), proved by the same argument.
- R_{IS} has no policy (φ) arm: its equations pin the witness (x₀, u) completely, so its proof is statement-independent (`SpeciallySound`, every statement). R_{IU}'s φ arm is not checked by the verification equation, which is why its theorem is per-statement, under `Enforces`.

Credential Presentation · R_p relation



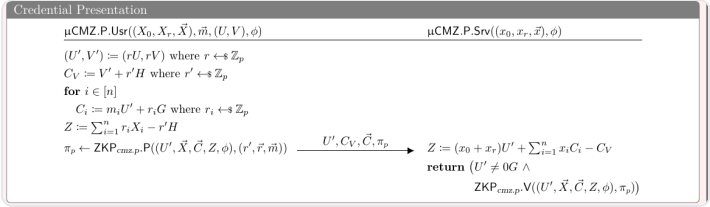
R_p — the user's proof: openings of the C_i and Z, policy φ

$$R_{\text{cmz.p}} := \left\{ \begin{array}{l} ((U', X_1, \dots, X_n, C_1, \dots, C_n, Z, \phi), (r', \vec{r}, \vec{m})) : \\ (\forall i \in [n] \ C_i = m_i U' + r_i G) \wedge Z = \sum_{i=1}^n r_i X_i - r' H \wedge \phi(\vec{m}) = 1 \end{array} \right\} \cdot \quad (11)$$

```
def rpRel (gen H : G) : RpStmt G F n → RpWitness F n → Bool :=
  fun ⟨U, X, C, Z, φ⟩ ⟨r', r, m⟩ => decide
    ((∀ i, C i = m i • U + r i • gen) ∧
     Z = (∑ i, r i • X i) - r' • H) && φ m
```

- **R_p in the box.** The user re-randomizes the credential σ = (U, V) into (U', V') = (r•U, r•V), commits to each attribute as C_i, masks the tag as C_V, and proves R_p (ZKP_{cmz.p.P}) with witness (r', r, m); the server checks U' ≠ 0 and the proof.
- **The two Z's.** The user computes Z = ∑_i r_i•X_i – r'•H from its randomness; the server recomputes Z = (x₀+x_r)•U' + ∑_i x_i•C_i – C_V from its key. The two agree exactly when V' = (x₀+x_r+∑_ix_im_i)•U', so an accepting proof convinces the server the presented commitments open to a validly MAC'd m, with σ never revealed.

Credential Presentation · Special Soundness



The explicit extractor (from `rpSigma`)

```
extract c1 z1 c2 z2 := pure
  ((z1.1 - z2.1) * (c1 - c2)-1,
   fun i => (z1.2.1 i - z2.2.1 i) * (c1 - c2)-1,
   fun i => (z1.2.2 i - z2.2.2 i) * (c1 - c2)-1)
```

Our theorem: special soundness with policy enforcement

```
theorem rpSigma_speciallySoundAt (gen H : G) (Up : G) (X : PublicBases G n)
  (C : Fin n → G) (Z : G) (φ : Policy F n)
  (hφ : Enforces (rpSigma (F := F) (n := n) gen H) (Up, X, C, Z, φ)
   (fun (_r', _r, m) => φ m)) :
  SpeciallySoundAt (rpSigma (F := F) (n := n) gen H) (Up, X, C, Z, φ)
```

- Same notion, same shape as [Issuance](#): VCV-io's perfect `SpeciallySoundAt`, an `Enforces` hypothesis for ϕ (discharged for `trivialPolicy` by `rp_enforces_trivialPolicy`), and a Schnorr extractor run once per verification equation.
- At R_p the extracted witness (r', r, m) opens every C_i and the Z equation, so a prover convincing the verifier knows the attributes m behind the presented commitments, satisfying ϕ .

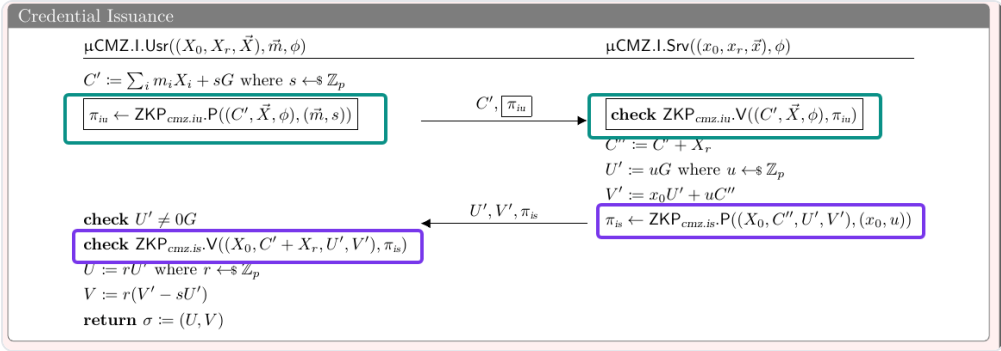
Status · Base MAC

Base MAC		
Procedure $\mu\text{CMZ}.S(1^\lambda, n)$ $(\mathbb{G}, p, G) := \Gamma \leftarrow \text{GrGen}(1^\lambda)$ $H \leftarrow \mathbb{S} \mathbb{G}$ return $\text{crs} := (\Gamma, H)$	Procedure $\mu\text{CMZ}.K(\text{crs})$ $sk := (x_0, x_r, x_1, \dots, x_n) \leftarrow \mathbb{S} \mathbb{Z}_q^{n+2}$ $X_0 := x_0 H; \quad X_r := x_r G$ $X_i := x_i G \quad \text{for } i \in [n]$ return $sk, pp := (X_0, X_r, \vec{X})$	Procedure $\mu\text{CMZ}.M((x_0, x_r, \vec{x}), \vec{m})$ $U \leftarrow \mathbb{S} \mathbb{G}$ $V := (x_0 + x_r + \sum_i m_i x_i) U$ return $\sigma := (U, V)$ Procedure $\mu\text{CMZ}.V((x_0, x_r, \vec{x}), \vec{m}, (U, V))$ return $U \neq 0G \wedge V = (x_0 + x_r + \sum_i m_i x_i) U$

- ✓ The four procedures and `macScalar` , packaged with perfect correctness as an instance of the `AlgebraicMAC` structure (`μCMZBaseMAC`).
- ✓ The hardness assumptions O24 needs beyond VCV-io's DL — 3-DL and 2-DL as q-DL instances, and gap-DL — in `Assumptions.lean` .
- ✓ Theorem 5.1's statement machinery in the AGM (algebraic adversary, representation-logging oracles with the Help oracle, game, advantage) with its polynomial toolkit (`AGMPolynomial`).
- ▢ The Theorem 5.1 reduction, n = 1 case (module `AGMReduction` , in review as PR #88).
- ▢ The n > 1 to n = 1 step (via gap-DL).
- ▢ The bridge from the AGM game to the plain UF-CMVA game of `KVAC.Core` .

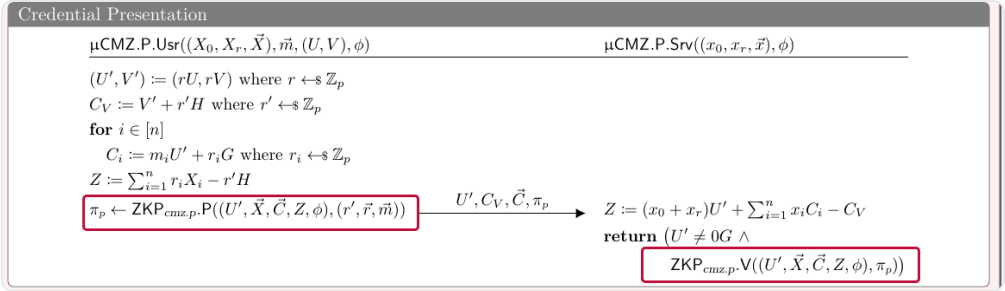
✓ formalized, merged on main · ▢ missing or still in review.

Status · Credential Issuance



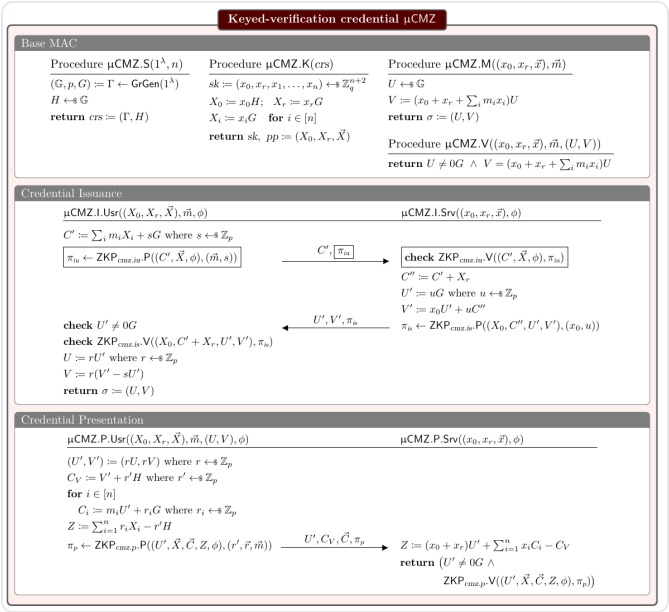
- ✓ Both proof relations of the box (`riuRel` , `risRel`).
- ✓ Their interactive Σ -protocols (`riuSigma` , `risSigma`): perfect completeness, special soundness, and HVZK via explicit simulated transcripts.
- ✓ The policy layer (`Policy` , `trivialPolicy`).
- ▢ The flow itself (blinding s , $C' \rightarrow C'' \rightarrow (U', V')$, unblinding $U := r \cdot U'$, $V := r(V' - s \cdot U')$) — it instantiates the abstract KVC syntax of Definition 4.2, whose framework layer is in review (PR #77); the μCMZ instantiation is not yet in a PR.
- ▢ Fiat–Shamir compilation of the Σ -protocols into the non-interactive π_{iu} , π_{is} .
- ▢ Credential-level correctness (Definition 4.3).

Status · Credential Presentation



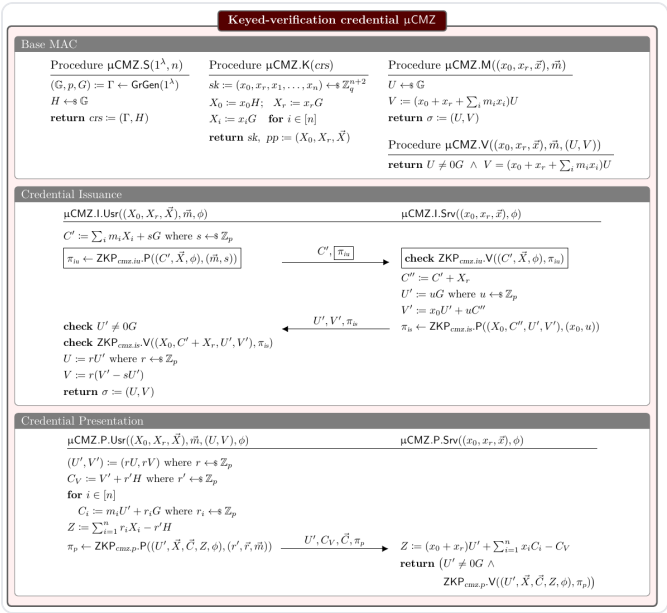
- ✓ The box's proof relation (`rpRel` , Eq. 11).
- ✓ Its interactive Σ -protocol `rpSigma`: perfect completeness (`rpSigma_complete`), special soundness with policy enforcement (`rpSigma_speciallySoundAt`), and HVZK via an explicit simulated transcript (`rpSigma_hvzk` , `rpSimTranscript`). The policy layer is shared with Issuance.
- ▢ The flow itself (re-randomization $(U', V') = (r \cdot U, r \cdot V)$, the masked tag C_V , the server-side Z recomputation), on the same Definition 4.2 track as Issuance (framework layer in review, PR #77).
- ▢ Fiat–Shamir compilation into π_p .
- ▢ The credential-level guarantees that consume it (anonymity and extractability, Theorem 5.2).

Design decisions · the model



- **Per-group, not GrGen.** Every theorem is an advantage inequality at one fixed group.
 - Γ becomes the typeclass plus `gen` ; the crs reduces to H; secParam and n are phantom arguments.
 - Why: it is the form O24's proofs natively establish, it needs no polynomial-time cost model, and it is what a deployment instantiates ([full analysis](#)).
- **Perfect correctness, not statistical.** Honest tags verify with probability exactly 1.
 - O24 is itself split: its defining equation samples U from the nonzero elements of $\mathbb{G}$, while the figures write $U \leftarrow \$ \mathbb{G}$, under which the honest tag (0, 0) fails verification with mass 1/p.
 - The Lean formalization follows the defining equation (`uniformNonzero`) and proves perfect (support-based) correctness.
 - The cost: the signing distribution is conditioned, and the reduction's counting argument under that law is deferred (`SignMask`).
- **Encodings, not semantics.** Representations a proof checker can compute with, and names that cannot collide.
 - Vectors are functions `Fin n → F` : length pinned by the type, equality pointwise.
 - Relations are Bool-valued via `decide` , so a machine can evaluate them.
 - The Σ -protocol's classical "commitment" is renamed announcement, avoiding a clash with the commitment C' .

Design decisions · the proofs



- **Interactive Σ -protocols now, Fiat–Shamir later.**
 - The figure's non-interactive $\pi_{\text{iu}}, \pi_{\text{is}}, \pi_{\text{p}}$ are not yet formalized; their underlying interactive Σ -protocols are, with perfect completeness, special soundness (policy-conditional, next bullet), and HVZK.
 - The compilation layer (random oracle, domain separation, knowledge extraction with its forking-lemma loss) is pending.
 - Why: the perfect properties are proven once at the Σ level and consumed by the compiled proofs.
- **Perfect special soundness, policy as a hypothesis.**
 - VCV-io's `SpeciallySoundAt` is transcript-level and λ -free, the information-theoretic counterpart of DG23's computational notion (Definition 4.6).
 - The ϕ arm is an `Enforces` hypothesis, discharged today only for `trivialPolicy`; a proper ϕ needs a combined proof binding ϕ to the same extracted witness.
 - Why: per-group there is no λ to be negligible in, and `verify` checks only the linear arm.
- **AGM baked into the unforgeability game.**
 - The game `AGM_UF_CMVAGame` instruments Figure 5 with two distinct changes: representation logging (a restriction to algebraic adversaries) and the Help oracle.
 - The Help oracle follows O24's proof preamble: unforgeability is proven in the stronger form where the adversary also gets Help, at no cost to the bound, because Theorem 5.11 ($\mu\text{CMZ}_{\text{AT}}$ one-more unforgeability) consumes exactly that stronger claim.
 - Its oracles are gated on representation consistency, so it matches the honest game only for well-behaved adversaries; the `WellBehaved` bridge is deferred.
 - Why: Theorem 5.1's proof is an AGM argument, and Lean must quantify over the algebraic adversary explicitly.

Recovering the O24-level statements is tracked work, not a given: averaging over GrGen for the model, the conditioned-distribution argument for the signing law, Fiat–Shamir compilation for the π 's, and the well-behaved bridge for the AGM game.

Up next

Theorem 5.1. *In the algebraic group model, μCMZ is an algebraic MAC for $n = \text{poly}(\lambda)$ attributes over $\mathbb{Z}_p$ with advantage:*

$$\text{Adv}_{\mu\text{CMZ}}^{\text{ufcmva}}(\lambda, n) \leq \text{Adv}_{\text{GrGen}}^{3\text{-dl}}(\lambda) + \text{Adv}_{\text{GrGen}}^{\text{dl}}(\lambda) + \frac{3}{p} ,$$

Theorem 5.2. *If ZKP is a proof system for the relation $R \supseteq R_{\text{cmz}}$, then μCMZ is a keyed-verification extractable credential for $n = \text{poly}(\lambda)$ attributes with anonymity advantage:*

$$\text{Adv}_{\mu\text{CMZ}}^{\text{anon}}(\lambda, n) \leq \text{Adv}_{\text{ZKP}_{\text{cmz.iu}}}^{\text{zk}}(\lambda) + \text{Adv}_{\text{ZKP}_{\text{cmz.p}}}^{\text{zk}}(\lambda) + \text{Adv}_{\text{ZKP}_{\text{cmz.is}}}^{\text{ksnd}}(\lambda) ,$$

and extractability advantage:

$$\text{Adv}_{\mu\text{CMZ}}^{\text{ext}}(\lambda, n) \leq \text{Adv}_{\mu\text{CMZ}}^{\text{ufcmva}}(\lambda, n) + \text{Adv}_{\text{ZKP}_{\text{cmz.is}}}^{\text{zk}}(\lambda) + \text{Adv}_{\text{ZKP}_{\text{cmz.iu}}}^{\text{ksnd}}(\lambda) + \text{Adv}_{\text{ZKP}_{\text{cmz.p}}}^{\text{ksnd}}(\lambda) .$$

Theorem 5.3. *If ZKP is a proof system for the relation $R \supseteq R_{\text{cmz.p}} \cup R_{\text{cmz.is}}$, then $\mu\text{CMZ}_{\text{AT}}$ is an anonymous token for $n = \text{poly}(\lambda)$ attributes with anonymity advantage:*

$$\text{Adv}_{\mu\text{CMZ}_{\text{AT}}}^{\text{anon}}(\lambda, n) \leq \text{Adv}_{\text{ZKP}_{\text{cmz.iu}}}^{\text{zk}}(\lambda) + \text{Adv}_{\text{ZKP}_{\text{cmz.p}}}^{\text{zk}}(\lambda) + \text{Adv}_{\text{ZKP}_{\text{cmz.is}}}^{\text{ksnd}}(\lambda) ,$$

and (in the algebraic group model) one-more unforgeability advantage:

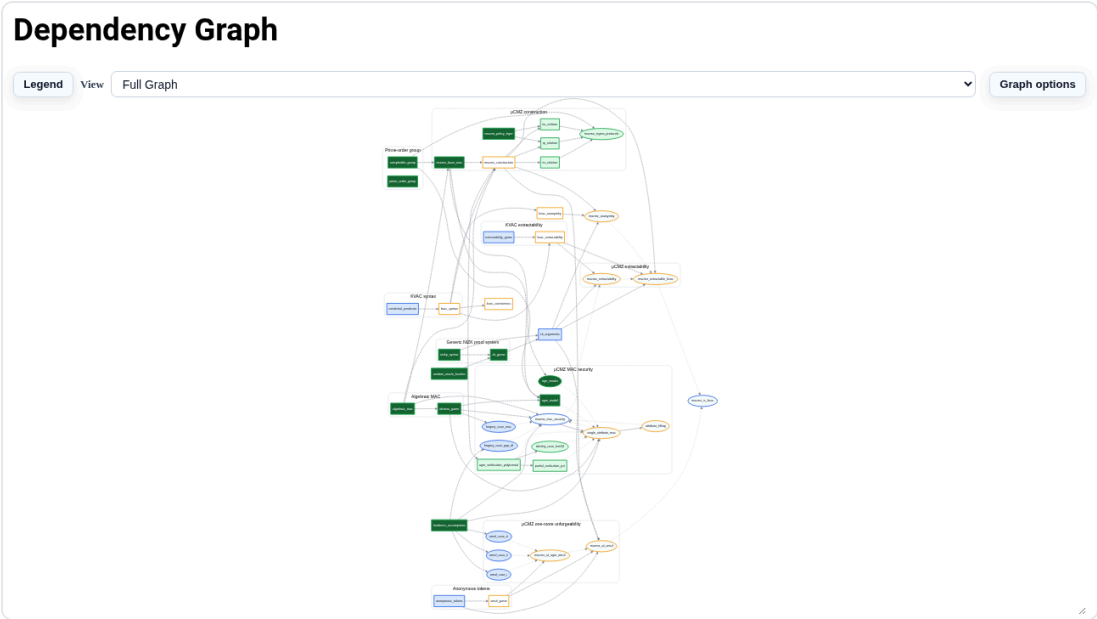
$$\text{Adv}_{\mu\text{CMZ}_{\text{AT}}}^{\text{omuf}}(\lambda, n) \leq \text{Adv}_{\text{GrGen}}^{2\text{dl}}(\lambda) + q \text{Adv}_{\text{GrGen}}^{\text{dl}}(\lambda) + \frac{q+5}{p} ,$$

where q is the number of queries to the issuance oracle.

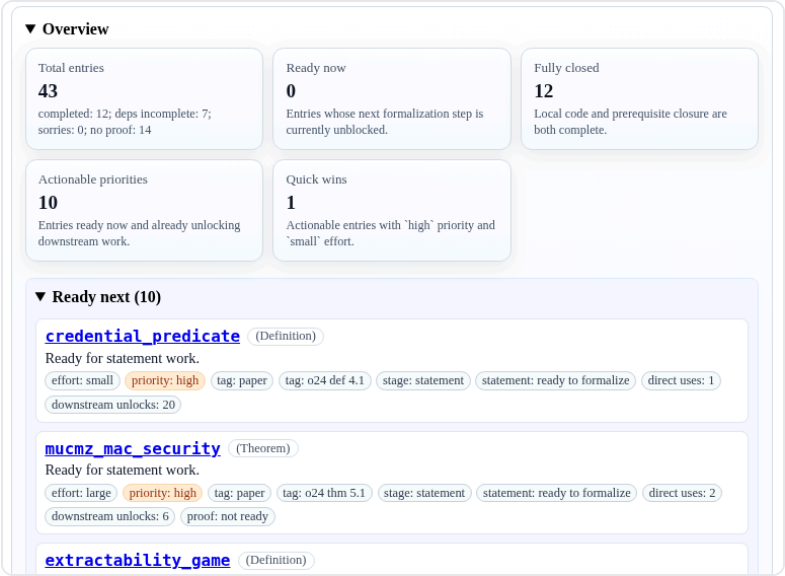
- **Now · Theorem 5.1 (Base MAC unforgeability).** The reduction core — Lemma 5.4's $n = 1$ case — is in review (module `AGMReduction`, PR #88). Behind it, in order: the bad-event probability bound, the $n > 1$ to $n = 1$ step (Lemma 5.5, with Claim 5.6's gap-DL reduction), and the bridge from the [AGM game](#) to the plain UF-CMVA game. Together these close Theorem 5.1.
- **Next · Theorem 5.2 (μCMZ is a keyed-verification credential).** Anonymity and extractability, conditional on a proof system for $R \supseteq R_{\text{cmz}}$. Its hypotheses are already in flight: knowledge soundness and simulation extractability of the NIZK layer (PR #54), the abstract credential framework the flows will instantiate (Definitions 4.1–4.3, PR #77), and Fiat–Shamir compilation of the merged Σ -protocols.
- **Then · Theorem 5.3 ($\mu\text{CMZ}_{\text{AT}}$, the anonymous token).** One-more unforgeability and anonymity of the token variant. It follows from Theorem 5.2's machinery plus the AGM analysis of Theorem 5.11; the assumption side (2-DL as `twoDlogAdv`) is merged.

The three theorems, stated at left as in O24 (p. 35), are the security story of the paper's Section 5 — Base MAC unforgeability (5.1), the full credential (5.2), the issuance-only token (5.3).

Tracking progress

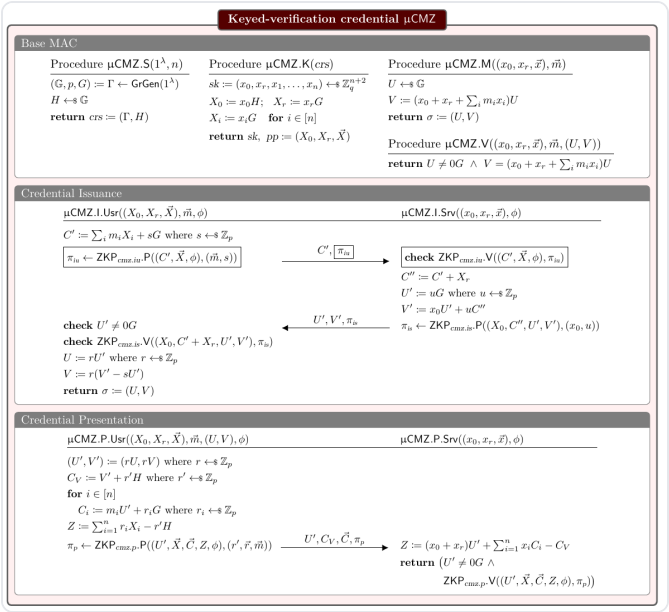


The blueprint summary, same site



- **Where.** The formalization publishes its own status page: beneficial-ai-foundation.github.io/KeyedVerificationAnonymousCredential-model. It rebuilds and redeploys on every merge, so it is never older than the last merged PR.
- **Dependency graph.** One node per definition and theorem of O24 plus our internal milestones, grouped by chapter, edges following the paper's dependencies; node color shows formalization status (the Legend button decodes it). The μ CMZ cluster mirrors the Section 5 story of these slides.
- **Blueprint summary.** The same nodes as a triaged list: overview counters (today, 43 tracked entries, 12 fully closed), then a ready-next queue with effort and priority tags.
- **Formalization progress tracker.** A generated per-element map of the whole paper, [FORMALIZATION_PROGRESS.md](#): every definition, theorem, lemma, figure, and equation of O24 with its formalization status and the Lean declarations citing it, with per-kind and per-section coverage tables.

Conclusion



- **All three boxes have machine-checked cores.** Base MAC: construction and perfect correctness ($\mu\text{CMZBaseMAC}$). Issuance and Presentation: their proof relations and interactive Σ -protocols, each with perfect completeness, special soundness, and HVZK.
- **Theorem 5.1 is underway.** The AGM statement machinery and its polynomial toolkit are merged; the $n = 1$ reduction core is in review (PR #88), with the $n > 1$ step and the game bridge behind it ([up next](#)).
- **The model is per-group.** Concrete advantage inequalities at a fixed group: the form O24's proofs natively establish, and the one a deployment instantiates ([setup](#), [analysis](#)).
- **Every departure from the paper is deliberate and disclosed** ([design decisions](#)), and the formalization already pays for itself in scrutiny: the $\mathbb{G}$ vs $\mathbb{G}^\times$ sampling split, Figure 9's $\mathbb{Z}_q$, and Lemma 5.4's constant all surfaced while preparing it.
- **Progress is public and CI-enforced.** The blueprint site rebuilds on every merge; every Lean declaration anchors to a blueprint node ([tracking](#)).